

ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 1

ОСНОВЫ PYTHON: АКТУАЛИЗАЦИЯ ЗНАНИЙ И НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

ЦЕЛЬ РАБОТЫ

Актуализировать базовые знания и навыки программирования на Python, подготовить рабочее окружение для разработки приложений и начать работу над индивидуальным сквозным проектом.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ	2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ	2
3. МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА	4
4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	4
4.1. Python в разработке серверных приложений.....	5
4.2. Рабочее окружение.....	5
4.3. Git.....	6
4.4. README.md.....	6
5. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА	6
5.1. Раздел 01. Основы Python.....	6
5.2. Раздел 02. Настройка рабочего окружения.....	7
6. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ	8
6.1. Общие положения и план развития проекта	8
6.2. Выбор темы проекта	9
6.3. Создание репозитория проекта.....	9
6.4. Подготовка файла README .md	10
6.5. Начальная реализация проекта	10
6.6. Отладка программы	11
6.7. Фиксация результата в Git.....	12
7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ.....	12
8. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ	13

8.1. Чек-лист выполнения материалов Яндекс Практикума	13
8.2. Критерии оценки	13
8.3. Требования к отчету	14
9. КОНТРОЛЬНЫЕ ВОПРОСЫ	15
10. ИТОГ	16

1. ЗАДАЧИ РАБОТЫ

В ходе выполнения практической работы необходимо:

- повторить базовые конструкции языка Python;
- настроить рабочее окружение для дальнейшей разработки;
- выполнить работу с системой контроля версий Git;
- проверить соответствие кода основным требованиям оформления;
- освоить базовые средства отладки программ;
- выбрать тему индивидуального сквозного проекта;
- создать репозиторий проекта;
- подготовить файл README.md;
- реализовать начальный программный сценарий индивидуального проекта;
- представить результаты работы преподавателю и ответить на контрольные вопросы.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

1. Войдите в личный аккаунт Яндекс с использованием персональной Яндекс-почты.
2. Подключитесь к курсу «Бэкенд-разработка» на образовательной платформе Яндекс Практикума по предоставленной преподавателем ссылке или приглашению.
3. *Ознакомьтесь с содержанием практической работы.* Перед началом работы ознакомьтесь:
 - с целью и задачами практической работы;
 - с перечнем материалов Яндекс Практикума, которые необходимо выполнить;
 - с требованиями к индивидуальному проекту;
 - с требованиями к отчету;
 - с формой контроля и перечнем контрольных вопросов.
4. *Выполните материалы Яндекс Практикума.* Выполните указанные в разделе «Материалы Яндекс Практикума» темы:

Раздел 01. Основы Python:

- Погружение в Python;
- Простые типы данных;
- Ветвления;
- Импортируемые типы данных.

Раздел 02. Настройка рабочего окружения:

- Настройка рабочего окружения;
- Pytest;
- Система контроля и управления версиями;
- Требования к коду;
- Отладка программ.

Отметьте выполненные материалы в чек-листе практической работы.

Темы «Коллекции», «Итерации и циклы» и проект «Холодильник» относятся к ПР2 и на ПР1 не выполняются.

5. *Выберите тему индивидуального проекта.* Выберите тему из предложенного перечня либо предложите собственную. Согласуйте выбранную тему с преподавателем. При выборе темы убедитесь, что она позволяет в дальнейшем реализовать полноценное веб-приложение на Django, работу с базой данных, пользователей, API и другие компоненты, предусмотренные программой дисциплины.
6. *Создайте Git-репозиторий индивидуального проекта.* Создайте отдельный репозиторий проекта. Подготовьте базовую структуру:

```
project-name/  
├── README.md  
├── .gitignore  
└── main.py
```

Настройте виртуальное окружение и проверьте запуск программы. Зафиксируйте начальное состояние проекта в Git.

7. *Заполните файл README.md.* В файл README.md внесите:

- название проекта;
- назначение;
- целевую аудиторию;
- описание предметной области;
- основные сущности;
- основные функции;
- план развития проекта.

README.md является частью индивидуального проекта и будет дополняться в последующих практических работах.

8. *Разработайте начальную версию индивидуального проекта.* Реализуйте один небольшой законченный сценарий, соответствующий выбранной предметной области. Используйте конструкции Python, изученные в рамках ПР1:

- простые типы данных;
- операции;

- преобразование типов;
- ветвления;
- импорт модулей.

На данном этапе не требуется реализовывать работу со сложными структурами данных и классами. Эти возможности будут добавлены при выполнении ПР2.

9. *Выполните отладку программы.* С использованием отладчика:

- установите точку останова;
- запустите программу в режиме отладки;
- выполните программу пошагово;
- проследите значения переменных;
- определите причину заранее внесенной ошибки;
- исправьте ошибку.

8. *Зафиксируйте результат в Git.* После завершения разработки:

- проверьте состояние репозитория;
- добавьте изменения;
- создайте commit;
- проверьте историю изменений;
- при использовании удаленного репозитория отправьте изменения.

В репозитории должен быть сохранен результат выполнения ПР1.

3. МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА

Практическая работа выполняется с использованием курса «Бэкенд-разработчик» Яндекс Практикума. Материалы курса выступают основным учебным ресурсом: теория, примеры и задания оформлены в виде последовательных тем, каждая из которых содержит практические упражнения с автоматической проверкой.

В рамках ПР1 используются два раздела курса:

- Раздел 01. Основы Python;
- Раздел 02. Настройка рабочего окружения.

Примечание. Материалы Яндекс Практикума используются как основной учебный ресурс. Дополнительные материалы настоящих методических указаний предназначены для организации практической работы, выполнения индивидуального проекта и контроля результатов.

4. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Перед выполнением практической части рекомендуется ознакомиться с краткими теоретическими сведениями. Они помогают восстановить контекст и понять, зачем выполняется каждый этап работы: от повторения конструкций языка до настройки системы контроля версий.

4.1. Python в разработке серверных приложений

Python является языком программирования общего назначения, широко используемым для разработки серверных приложений. В дальнейшем в рамках дисциплины Python будет использоваться совместно с фреймворком Django, поэтому уверенное владение базовыми конструкциями языка является обязательным условием успешного освоения последующих тем.

Для дальнейшей работы необходимо уверенно владеть базовыми конструкциями языка:

- переменными;
- основными типами данных;
- операциями;
- преобразованием типов;
- условными конструкциями;
- импортом модулей;
- коллекциями;
- циклами;
- функциями;
- объектно-ориентированным программированием.

ПР1 посвящена актуализации первых базовых элементов. Работа с коллекциями, функциями, циклами и проектом «Холодильник» выполняется на ПР2.

4.2. Рабочее окружение

Рабочее окружение включает набор программных средств, необходимых для разработки и проверки программ. От корректности его настройки зависит выполнение всех последующих работ дисциплины, поэтому настройке окружения уделяется отдельное внимание уже на первом этапе.

В рамках дисциплины используются:

- Python;
- IDE;
- виртуальное окружение;
- Git;
- удаленный Git-репозиторий;
- средства отладки.

Подготовленное на ПР1 окружение будет использоваться на последующих практических работах, поэтому все компоненты необходимо настроить и проверить до перехода к индивидуальному проекту.

4.3. Git

Git предназначен для управления версиями исходного кода. С его помощью фиксируются изменения, восстанавливается история разработки и организуется работа с проектом на протяжении всей дисциплины.

В рамках практической работы используются основные операции:

- создание репозитория;
- добавление файлов;
- фиксация изменений (`commit`);
- просмотр истории изменений;
- работа с удаленным репозиторием.

Система контроля версий будет использоваться при разработке индивидуального проекта на протяжении всей дисциплины, поэтому базовые операции Git должны быть освоены именно на ПР1.

4.4. README.md

Файл `README.md` содержит основную информацию о проекте и является стандартным элементом программного репозитория. Именно с него начинается знакомство любого разработчика с проектом, поэтому качество его заполнения влияет на восприятие работы.

В рамках индивидуального проекта он будет постепенно дополняться по мере развития приложения. На ПР1 в нем фиксируются:

- название проекта;
- назначение;
- целевая аудитория;
- предметная область;
- основные сущности;
- основные функции;
- план дальнейшего развития.

5. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

Практическая часть начинается с учебных материалов: сначала актуализируются знания языка и настраивается рабочее окружение, и только после этого выполняется индивидуальный проект (раздел 6). Не приступайте к подразделам 6.1–6.7, пока не завершены подразделы 5.1 и 5.2.

5.1. Раздел 01. Основы Python

На ПР1 выполняются только первые три темы раздела. Остальные темы переносятся на ПР2 – их выполнение на первом этапе не требуется.

№	Тема	Выполнить на ПР1
1	Погружение в Python	✓
2	Простые типы данных	✓
3	Ветвления	✓
4	Импортируемые типы данных	✓
5	Коллекции	—
6	Итерации и циклы	—
7	Проектная работа «Холодильник»	—

В рамках тем необходимо выполнить материалы Практикума, посвященные:

Простым типам данных:

- числовым типам;
- строкам;
- логическому типу;
- операциям над данными;
- преобразованию типов.

Ветвлениям:

- условной конструкции `if`;
- конструкциям `if-else` и `if-elif-else`;
- операторам сравнения;
- логическим операциям;
- составным условиям.

Импортируемым типам данных:

- подключению модулей;
- использованию функций и объектов импортируемых модулей;
- работе со стандартными модулями Python.

Важно. Темы «Коллекции», «Итерации и циклы» и проект «Холодильник» на ПР1 не выполняются. Они являются содержанием ПР2.

5.2. Раздел 02. Настройка рабочего окружения

Выполните все темы раздела 02 «Настройка рабочего окружения». Каждая тема завершается практическими действиями, результат которых понадобится при выполнении индивидуального проекта.

№	Тема	Выполнить на ПР1
1	Настройка рабочего окружения	✓
	Pytest	✓
2	Система контроля и управления версиями	✓
3	Требования к коду	✓
4	Отладка программ	✓

В результате необходимо:

- проверить установку Python;

- настроить IDE;
- создать виртуальное окружение;
- проверить запуск Python-программы;
- настроить Git;
- создать или клонировать репозиторий;
- выполнить базовые операции Git;
- проверить оформление кода;
- запустить программу в режиме отладки;
- установить breakpoint и выполнить программу пошагово.

6. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

В течение двух семестров студент разрабатывает индивидуальное приложение, которое последовательно расширяется по мере изучения дисциплины. На ПР1 выполняется стартовая часть проекта:

- выбирается предметная область,
- создается репозиторий,
- готовится описание приложения
- реализуется первый программный сценарий.

Подразделы 6.1–6.7 отражают фактический порядок выполнения: переходите к следующему подразделу только после завершения предыдущего.

6.1. Общие положения и план развития проекта

Проект развивается по сквозной схеме: каждая практическая работа добавляет к приложению новые технологии и функциональность. План показывает, зачем на каждом этапе осваиваются соответствующие технологии и какой результат должен получить проект к завершению дисциплины.

ПР	Основные технологии и темы	Результат развития ИПа
ПР1	Основы Python, рабочее окружение, Git	Репозиторий проекта, настроенное окружение, README.md, начальный сценарий предметной области
ПР2	Коллекции, функции, циклы, модули, файлы, исключения, типизация, требования к коду	Консольная программа с несколькими сценариями, функциями и структурой модулей
ПР3	ООП, классы, объекты, наследование, полиморфизм, инкапсуляция	Основные сущности проекта реализованы в виде классов
ПР4	Контрольная работа № 1	Проверка освоения Python и ООП
ПР5	Основы веб-разработки, Django, HTTP	Создан Django-проект и базовая структура веб-приложения
ПР6	URL, view-функции, HTML, шаблоны Django	Реализовано отображение данных и основные веб-страницы
ПР7	Базы данных, SQL, модели Django, ORM	Созданы модели предметной области и подключена база данных
ПР8	Контрольная работа № 2	Проверка основ Django, HTTP и работы с

ПР	Основные технологии и темы	Результат развития ИПа
		моделями
ПР9	Django ORM, запросы, связи моделей, админ-зона	Реализована полноценная работа с данными и административный интерфейс
ПР10	Django Forms, обработка данных	Реализованы формы создания и изменения данных
ПР11	Пользователи Django, регистрация, аутентификация, авторизация	Реализованы пользовательские сценарии и разграничение доступа
ПР12	Контрольная работа № 3	Проверка работы с Django, ORM, формами и пользователями
ПР13	Тестирование на Python, принципы и инструменты тестирования	Определены тестируемые компоненты и сценарии проекта
ПР14	Pytest и тестирование Django	Реализованы автоматизированные тесты
ПР15	Доработка и расширение тестирования	Исправление ошибок, повышение покрытия ключевых сценариев
ПР16	Контрольная работа № 4	Итоговая проверка материала первого семестра

6.2. Выбор темы проекта

Выберите тему из [предложенного перечня](#) либо предложите собственную. При выборе ориентируйтесь на предметную область, которая Вам понятна: проектировать сущности и функции проще для сервиса, привычного повседневному пользователю.

Выбранная предметная область должна позволять в дальнейшем:

- выделить несколько сущностей;
- хранить данные в базе данных;
- установить связи между сущностями;
- реализовать пользователей;
- реализовать операции создания, просмотра, изменения и удаления данных;
- создать веб-приложение на Django;
- разработать REST API;
- развернуть приложение в Docker.

Тема проекта должна быть согласована с преподавателем до перехода к подразделу 6.3.

6.3. Создание репозитория проекта

После согласования темы создайте отдельный Git-репозиторий для индивидуального проекта. Репозиторий – это «дом» проекта на весь срок дисциплины: в нем будут храниться код, описание и история разработки. На ПР1 рекомендуется следующая структура:

```
project-name/
├── README.md
├── .gitignore
└── main.py
```

Приведенная структура является минимальной и будет изменяться в процессе дальнейшей разработки по мере усложнения приложения.

6.4. Подготовка файла README.md

Создайте в корневой директории проекта файл README.md и заполните его по следующему шаблону:

```
# Название проекта

## Назначение

Краткое описание приложения и задачи, которую оно решает.

## Целевая аудитория

Описание предполагаемых пользователей.

## Предметная область

Краткое описание предметной области.

## Основные сущности

- ...
- ...
- ...

## Основные функции

- ...
- ...
- ...

## План развития

На следующих этапах планируется:

- разработка веб-приложения;
- подключение базы данных;
- реализация пользователей;
- разработка API;
- контейнеризация приложения;
- настройка CI/CD.
```

Примечание. В начальном варианте необходимо указать не менее 3-х функций.

Содержание README.md должно соответствовать выбранной теме проекта: разделы шаблона заполняются конкретными формулировками, а не остаются с многоточиями.

6.5. Начальная реализация проекта

Разработайте один небольшой законченный сценарий выбранного приложения. Используйте конструкции Python, рассмотренные в рамках ПР1:

- простые типы данных;
- операции;
- преобразование типов;
- условные конструкции;
- функции;
- импорт модулей.

Примечание. На ПР1 не требуется реализовывать сложную обработку наборов данных. Работа с коллекциями, классами, итерациями и циклами будет подробно закреплена на ПР2. При этом необходимо выполнить начальную реализацию не менее 3-х описанных в README.md функций

Пример начального сценария для проекта «Система бронирования помещений» – проверка возможности бронирования помещения:

```
from datetime import date

room_name = "Аудитория 301"
capacity = 30
booking_date = date(2026, 9, 15)
is_available = True

def get_booking_status(is_available):
    if is_available:
        return "Помещение доступно для бронирования"
    return "Помещение уже занято"

print(f"Помещение: {room_name}")
print(f"Вместимость: {capacity} человек")
print(f"Дата: {booking_date}")
print(get_booking_status(is_available))
```

Примечание. В примере реализована 1 функция, в ходе выполнения задания необходимо реализовать не менее 3-х описанных в README.md функций

Приведенный пример не является заданием для копирования. Реализуйте собственный сценарий, соответствующий выбранной теме проекта.

6.6. Отладка программы

Проверьте работу созданной программы с использованием отладчика IDE. Отладка – обязательный этап: она подтверждает, что программа работает именно так, как задумано. Необходимо:

- установить breakpoint;
- запустить программу в режиме отладки;
- выполнить программу пошагово;
- проследить значения переменных;
- определить значение переменной в выбранной точке выполнения.

После этого внесите небольшое изменение, приводящее к ошибочному результату, и найдите причину ошибки с использованием отладчика. Такое упражнение наглядно демонстрирует практическую пользу пошаговой отладки при поиске дефектов.

6.7. Фиксация результата в Git

После завершения работы зафиксируйте результат в репозитории:

- проверьте состояние рабочего дерева;
- добавьте необходимые файлы;
- создайте `commit`;
- проверьте историю изменений;
- отправьте изменения в удаленный репозиторий, если он используется.

Результат ПР1 должен быть полностью сохранен в репозитории: это завершает цикл разработки первого этапа индивидуального проекта и позволяет преподавателю проверить историю выполнения.

7. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

К завершению ПР1 должны быть получены следующие результаты. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтвержден фактическим результатом.

Материалы Яндекс Практикума

Раздел 01. Основы Python:

- выполнена тема «Погружение в Python»;
- выполнена тема «Простые типы данных»;
- выполнена тема «Ветвления»;
- выполнена тема «Импортируемые типы данных».

Раздел 02. Настройка рабочего окружения:

- выполнена тема «Настройка рабочего окружения»;
- выполнена тема «Pytest»;
- выполнена тема «Система контроля и управления версиями»;
- выполнена тема «Требования к коду»;
- выполнена тема «Отладка программ».

Индивидуальный проект

- выбрана тема;
- тема согласована с преподавателем;
- создан Git-репозиторий;
- создан `.gitignore`;

- создан и заполнен README.md;
- реализован начальный программный сценарий;
- выполнена отладка;
- результат сохранен в Git.

8. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Контроль выполнения ПР1 проводится индивидуально. Форма контроля включает:

- проверку выполнения материалов Яндекс Практикума;
- проверку рабочего окружения;
- демонстрацию индивидуального проекта;
- проверку репозитория;
- устный ответ на контрольные вопросы.

8.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов. Отмечайте пункты по мере выполнения – чек-лист используется и при текущем контроле, и при защите работы.

Чек-лист раздела 01 «Основы Python»

Материал	Выполнено
Погружение в Python	☐
Простые типы данных	☐
Ветвления	☐
Импортируемые типы данных	☐
Коллекции	ПР2
Итерации и циклы	ПР2
Проект «Холодильник»	ПР2

Чек-лист раздела 02 «Настройка рабочего окружения»

Материал	Выполнено
Настройка рабочего окружения	☐
Pytest	☐
Система контроля и управления версиями	☐
Требования к коду	☐
Отладка программ	☐

Таким образом, для допуска результата ПР1 к проверке должны быть отмечены все пункты, относящиеся к ПР1.

8.2. Критерии оценки

Работа считается выполненной, если по каждому показателю достигнут требуемый результат. Критерии приведены используются преподавателем при защите работы.

№	Показатель	Что должно быть продемонстрировано
1	Материалы Яндекс Практикума	все темы ПР1 из чек-листов выполнены; по запросу преподавателя показано конкретное выполненное задание или результат его выполнения
2	Рабочее окружение	Python, IDE, виртуальное окружение и Git установлены и настроены; продемонстрирован запуск Python-программы
3	Индивидуальный проект	тема согласована; репозиторий создан; README.md и .gitignore заполнены; начальный сценарий реализован и корректно работает
4	Отладка	продемонстрированы установка breakpoint, запуск в режиме отладки, пошаговое выполнение и просмотр значений переменных
5	Фиксация в Git	commit создан, история изменений просматривается, изменения отправлены в удаленный репозиторий (если используется)
6	Контрольные вопросы	даны содержательные ответы на вопросы по материалам Практикума, рабочему окружению и индивидуальному проекту

8.3. Требования к отчету

Отчет является документом, подтверждающим выполнение практической работы. Объем отчета – 1–2 *страницы* без учета титульной информации. В отчете необходимо указать:

1. Общие сведения:

- дисциплина;
- номер и тема практической работы;
- группа;
- ФИО студента;
- год выполнения.

2. Выполненные материалы Яндекс Практикума

Раздел	Выполненные темы
01. Основы Python	Погружение в Python, Простые типы данных; Ветвления; Импортируемые типы данных
02. Настройка рабочего окружения	Настройка рабочего окружения; Pytest; Система контроля и управления версиями; Требования к коду; Отладка программ

3. Индивидуальный проект. Указать:

- название проекта;
- выбранную предметную область;
- краткое описание реализованного на ПР1 функционала;
- ссылку на Git-репозиторий.

4. Результат работы. Кратко указать, что в результате выполнения ПР1:

- настроено рабочее окружение;
- создан репозиторий индивидуального проекта;
- подготовлен README.md;
- реализован начальный программный сценарий;
- выполнена отладка программы.

5. Вывод. Сделать краткий вывод о результатах выполнения практической работы.

9. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите используйте контрольные вопросы. Они сгруппированы по трем направлениям и охватывают все разделы работы: материалы Практикума, рабочее окружение и индивидуальный проект.

По материалам Яндекс Практикума

1. Какие основные типы данных используются в Python?
2. Чем отличаются `int`, `float`, `str` и `bool`?
3. Для чего используется преобразование типов?
4. Какие операции можно выполнять со строками?
5. Какие операторы сравнения используются в Python?
6. Для чего используется конструкция `if`?
7. В чем различие между `if`, `elif` и `else`?
8. Как записываются составные условия?
9. Для чего используются логические операторы `and`, `or` и `not`?
10. Что такое модуль Python?
11. Для чего используется `import`?
12. Чем импорт модуля отличается от импорта отдельного объекта из модуля?

По рабочему окружению

13. Для чего используется виртуальное окружение Python?
14. Что такое IDE?
15. Для чего используется Git?
16. Что такое репозиторий?
17. Что такое `commit`?
18. Для чего нужен `.gitignore`?
19. Для чего нужен `README.md`?
20. Что такое отладка?
21. Что такое `breakpoint`?
22. Для чего используется пошаговое выполнение программы?

По индивидуальному проекту

23. Какую задачу решает выбранное приложение?
24. Кто является предполагаемым пользователем?
25. Какие основные сущности выделены в предметной области?
26. Какие функции планируется реализовать?
27. Какой сценарий реализован на текущем этапе?
28. Какие конструкции Python использованы в программе?
29. Какие части проекта будут добавлены после изучения ООП и Django?

30. Почему выбранная предметная область подходит для дальнейшей реализации в виде веб-приложения?

10. ИТОГ

В результате выполнения ПР1 должна быть создана основа для дальнейшей разработки: актуализированные знания языка, настроенное окружение и стартовая версия индивидуального проекта.

На следующей практической работе продолжается раздел «Основы Python»: изучаются коллекции, функции, итерации и циклы, после чего выполняется проект «Холодильник». Параллельно индивидуальный проект получает более содержательную структуру данных и функциональность.